EXPERIMENT 15

Career Pathways and the Future of Low-Code Development

Industry-Wide Study — Report and Presentation Outline

Ananya P
241401010

1. Aim

To study the career paths available to low-code developers and the direction in which low-code and no-code development is heading across the industry, covering several platforms and not only Zoho Creator, and to prepare a report and a presentation outline that link these trends to the skills built in the earlier experiments.

The study helps decide which skills to learn next and which roles to work towards as low-code, pro-code and AI-assisted development continue to merge.

2. Procedure

Step 1: Review the history — The shift from manual coding to frameworks, cloud services, low-code platforms and AI-assisted building was laid out as a timeline (Section 3.1).

Step 2: Compare platforms — Popular low-code and no-code tools were compared by their strengths and typical use, to show that the field is far wider than one product (Section 3.2).

Step 3: Study the trends — Current industry trends such as AI copilots, autonomous agents, composable apps, governed citizen development and legacy modernisation were listed with their effect on jobs (Section 3.3), and the expected future workflow was drawn (Section 3.9).

Step 4: Map the careers — Roles were arranged from entry level to senior level, along with possible career switches, as a pathway diagram and a role table (Sections 3.4 and 3.5).

Step 5: Plan the skills — The skills required were grouped into layers and converted into a step-by-step learning roadmap that starts from the experiments already completed (Sections 3.6 and 3.7).

Step 6: Assess the risks — Common problems such as shadow IT, weak security, lock-in and errors in AI-generated logic were listed with ways to handle them (Section 3.8).

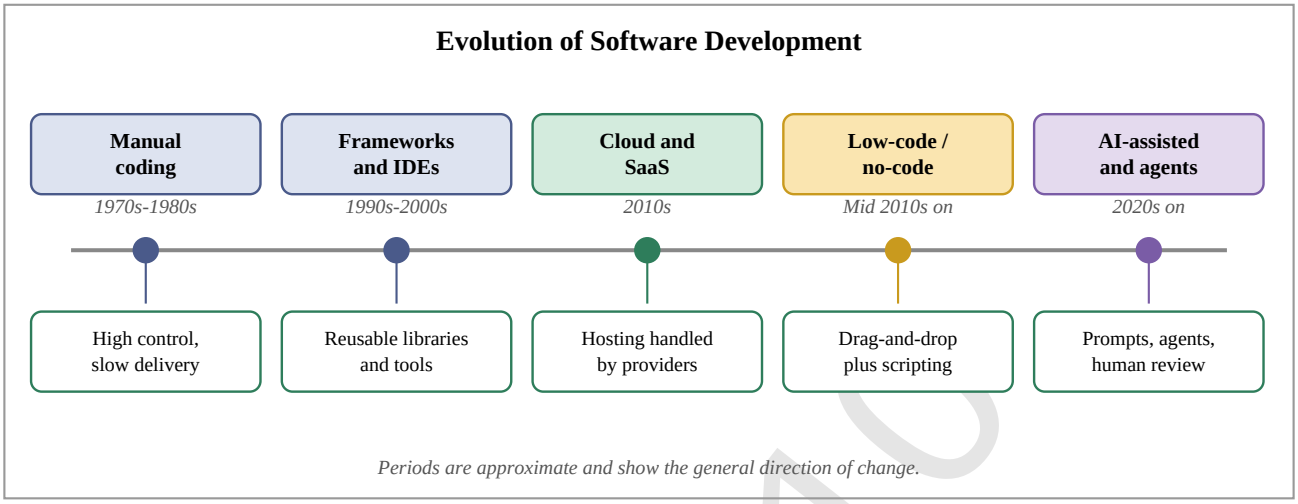
Step 7: Prepare the presentation — A slide-by-slide outline was written so that the findings can be presented clearly (Section 3.10).

Note: this is a study based on general industry knowledge and published articles. Forecasts and statistics found in such articles come from secondary sources and must be verified before they are quoted as facts. The timeline and roles are a general guide, not a prediction for any one company.

241401010

3. Output

3.1 Evolution of software development



The timeline shows the move from hand-written code to reusable frameworks, then cloud services, visual low-code platforms and now AI-assisted and agent-based building. Each stage reduced the effort needed to produce an application, while the need for design, testing and oversight remained.

3.2 Platform comparison

Low-code and no-code is a broad field. The platforms below serve different needs, and the core concepts (data models, forms, workflows, permissions, integrations) carry over from one to another.

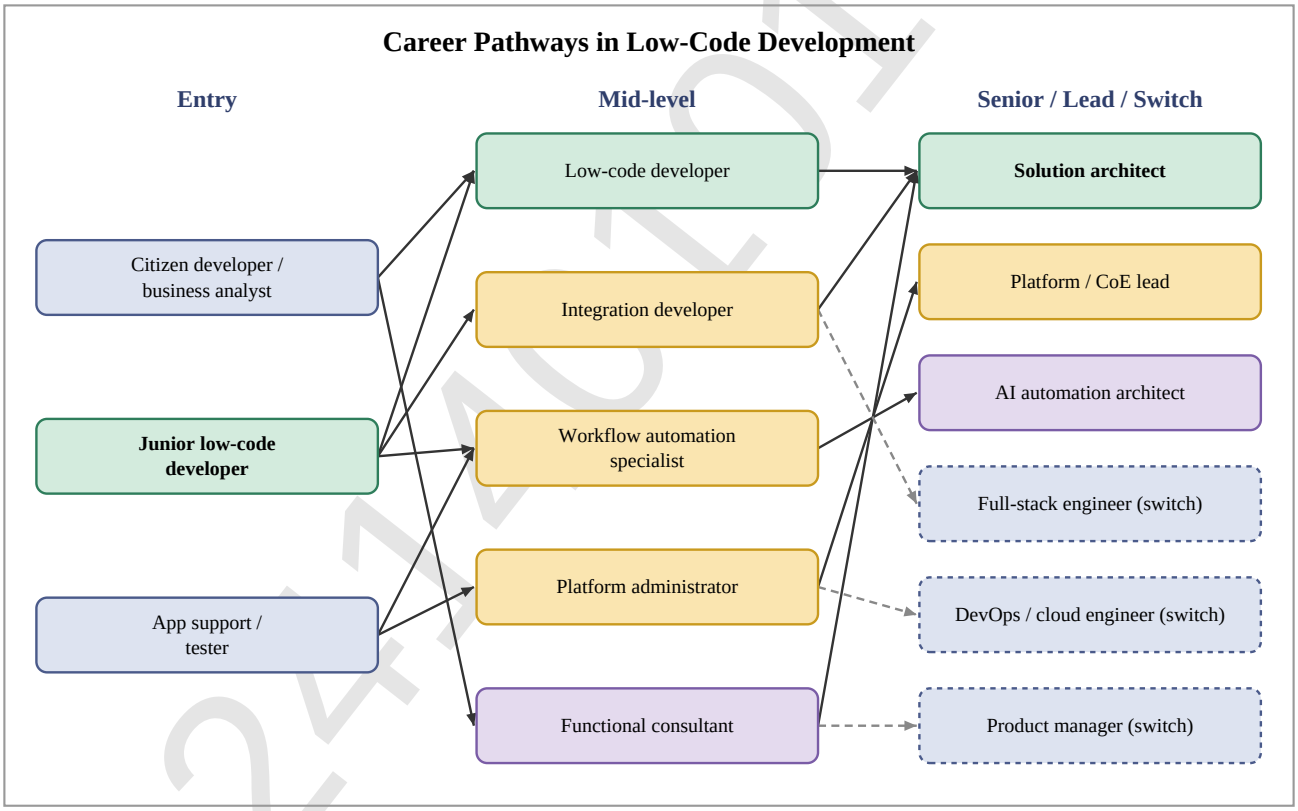
Platform	Best known for	Typical use
Zoho Creator	Custom business apps within the Zoho ecosystem.	Workflow and database apps for small and mid-size firms.
Google AppSheet	Apps built directly from spreadsheets and Google data.	Field data collection, inventory and simple approvals.
Airtable	Flexible database with automations and interfaces.	Project tracking and operations management.
Retool	Internal tools built on databases and APIs.	Admin panels and dashboards for technical teams.
Bubble	No-code web application building.	Startup prototypes and customer-facing web apps.
ServiceNow App Engine	Workflow apps on the ServiceNow platform.	IT service and enterprise workflow automation.
Pega	Case management and decision automation.	Banking, insurance and other regulated processes.

24AD101

3.3 Trends shaping the future

Trend	What it means	Effect on careers
AI copilots in builders	Prompts in plain language generate forms, fields and logic.	Less manual building, more reviewing and refining.
Autonomous AI agents	Agents carry out tasks inside workflows.	Demand for people who design and supervise agents.
API-first, composable apps	Apps are assembled from reusable services.	Integration skills become a core requirement.
Governed citizen development	Non-IT staff build apps under company rules.	Need for platform admins and standards owners.
Automation convergence	Low-code, RPA and AI are combined.	Automation engineers and process analysts are valued.
Legacy modernisation	Old systems get new low-code front ends.	Migration and integration work grows.
Security and compliance focus	More apps means more risk.	Governance and testing skills gain value.

3.4 Career pathways



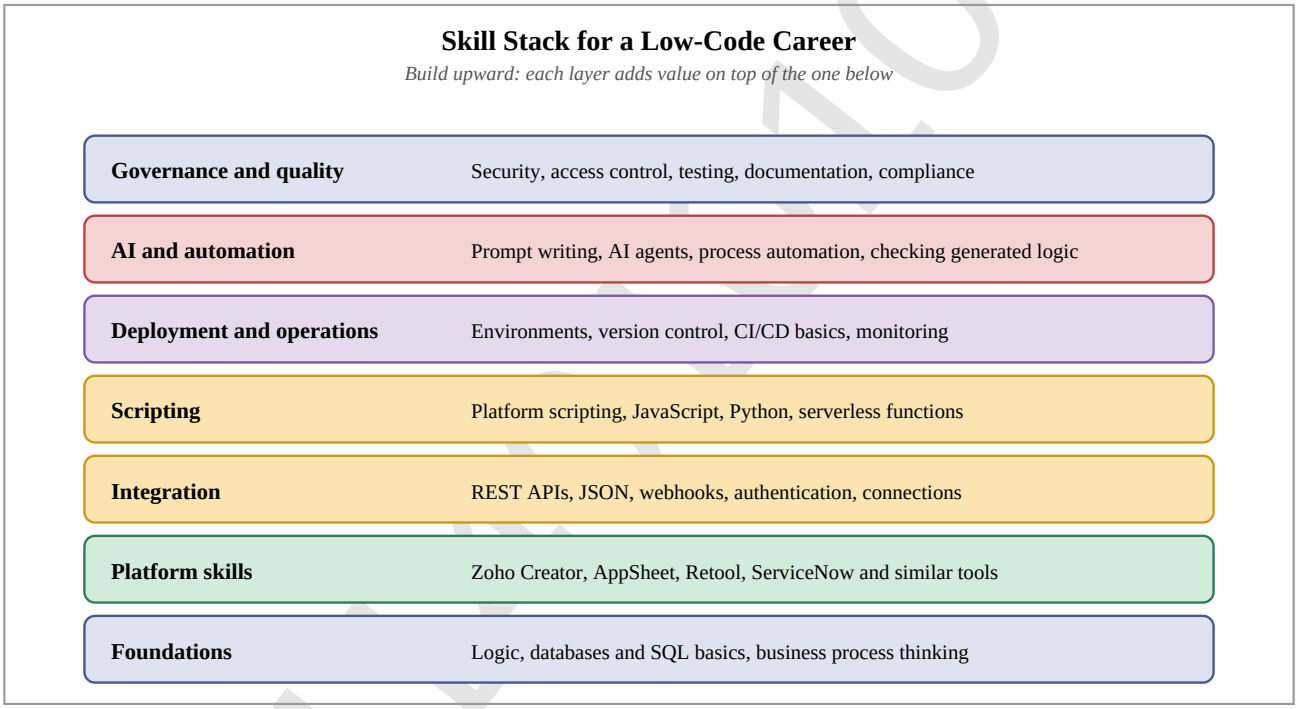
The diagram shows common routes from entry roles such as citizen developer, junior developer or app tester, through mid-level roles in development, integration, automation, administration and consulting, to senior roles such as architect, platform lead or AI automation architect. Dashed arrows and boxes show possible switches to full-stack engineering, DevOps and cloud, or product management.

24101010

3.5 Roles at a glance

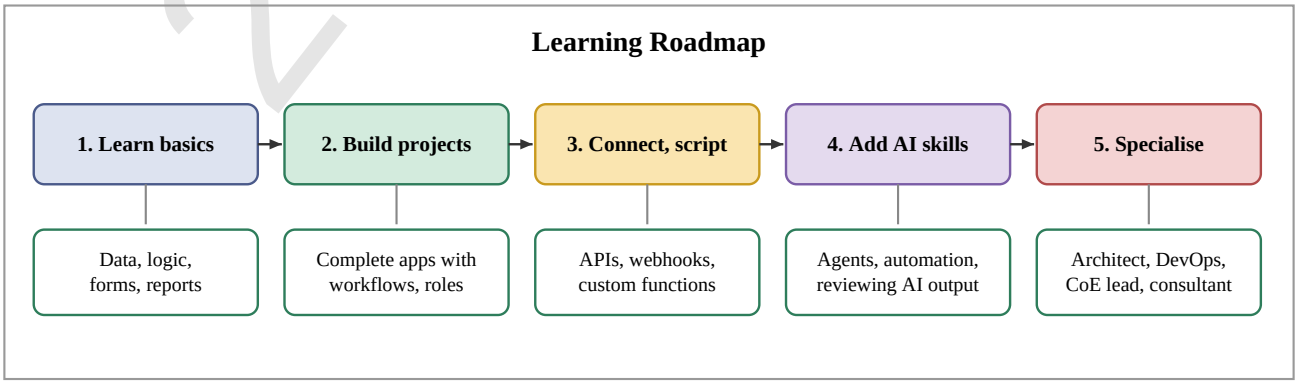
Role	What the person does	Key skills
Junior low-code developer	Builds forms, reports and basic workflows.	Platform basics, data modelling.
Low-code developer	Builds complete apps with custom scripting.	Scripting, workflow logic, UI design.
Integration developer	Connects apps to other systems through APIs and webhooks.	REST, JSON, authentication, error handling.
Automation specialist	Automates multi-step business processes, including AI steps.	Workflow design, RPA tools, AI services.
Platform administrator	Manages users, environments and releases.	Access control, deployment, monitoring.
Solution architect	Designs the overall structure, security and scale.	Architecture, governance, communication.

3.6 Skill stack for a low-code career



Skills are arranged in seven layers, from foundations such as logic and data up to governance. The upper layers are harder to automate and therefore carry the most long-term career value.

3.7 Learning roadmap

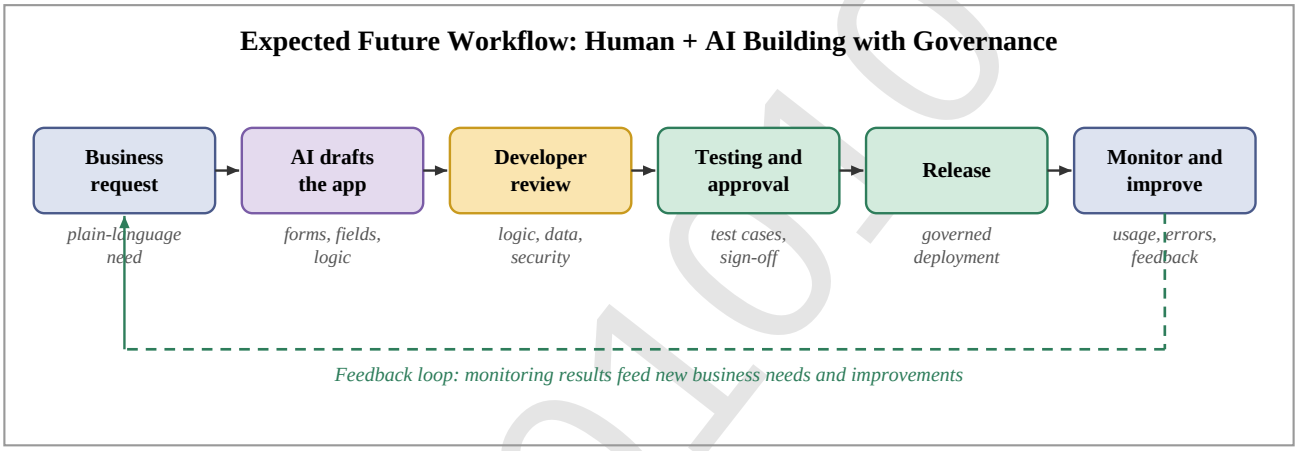


The roadmap moves from basics to projects, then to connections and scripting, AI and automation, and finally specialisation. The experiments completed so far form the base of steps one to three, and the next stages are deployment practice, AI-based automation and cloud fundamentals.

3.8 Risks and challenges

Challenge	Why it matters	How to handle it
Unmanaged apps (shadow IT)	Untracked apps cause security gaps and duplication.	Central app registry and approval process.
Weak security	Easy building can expose data.	Role-based access, reviews, audit logs.
Errors in AI-generated logic	Output can look right but be wrong.	Human review, test cases, version control.
Vendor lock-in	Apps depend on one vendor's tools and pricing.	Use open APIs and keep data exportable.
Scalability limits	Low-code apps may struggle under heavy load.	Plan architecture early; extend with pro-code.
Shifting job market	Routine build work may shrink as AI improves.	Move towards integration, design and oversight.

3.9 Expected future workflow



In the expected workflow a business request is described in plain language, AI generates a draft application, a developer reviews logic, data and security, the app is tested and approved, then released, and monitoring feeds new improvements back to the business. AI speeds up drafting, while humans remain responsible for review, approval and accountability.

3.10 Presentation outline

The findings can be presented in ten slides, one idea per slide:

Slide	Title	Key points
1	Title and introduction	Topic, name and roll number.
2	What is low-code?	Visual building with optional code; examples from my experiments.
3	Evolution of development	Timeline from manual coding to AI-assisted building.
4	Platform comparison	Zoho Creator, AppSheet, Airtable, Retool, Bubble, ServiceNow, Pega.
5	Future trends	AI copilots, agents, composable apps, governed citizen development.
6	Career pathways	Pathway diagram from entry to senior roles and switches.
7	Skills and roadmap	Skill stack and learning roadmap.
8	Risks and solutions	Shadow IT, security, AI output errors, lock-in.
9	My experience	Completed experiments as practical examples.
10	Conclusion and Q&A	Low-code and pro-code will work together; keep learning.

24AD010

4. Final Result

The study showed that low-code has grown from a quick way to build simple apps into a wider field where visual building, scripting and AI work together. Many platforms exist, and the core skills learned on one of them, such as Zoho Creator, carry over to the others.

Career paths run from citizen developer and junior developer roles, through integration, automation and administration, to architect and platform lead positions, with routes into engineering, cloud and product work. The skill stack and roadmap show how the earlier experiments form the first steps of this path.

The main risks are unmanaged apps, security gaps, errors in AI-generated output and vendor lock-in, and they are reduced through governance, testing and human review. The exercise met its objective, and the presentation outline is ready to be turned into slides. Since the conclusions rest on general knowledge and published articles, any statistics should be checked against the original sources before use.

24140101O